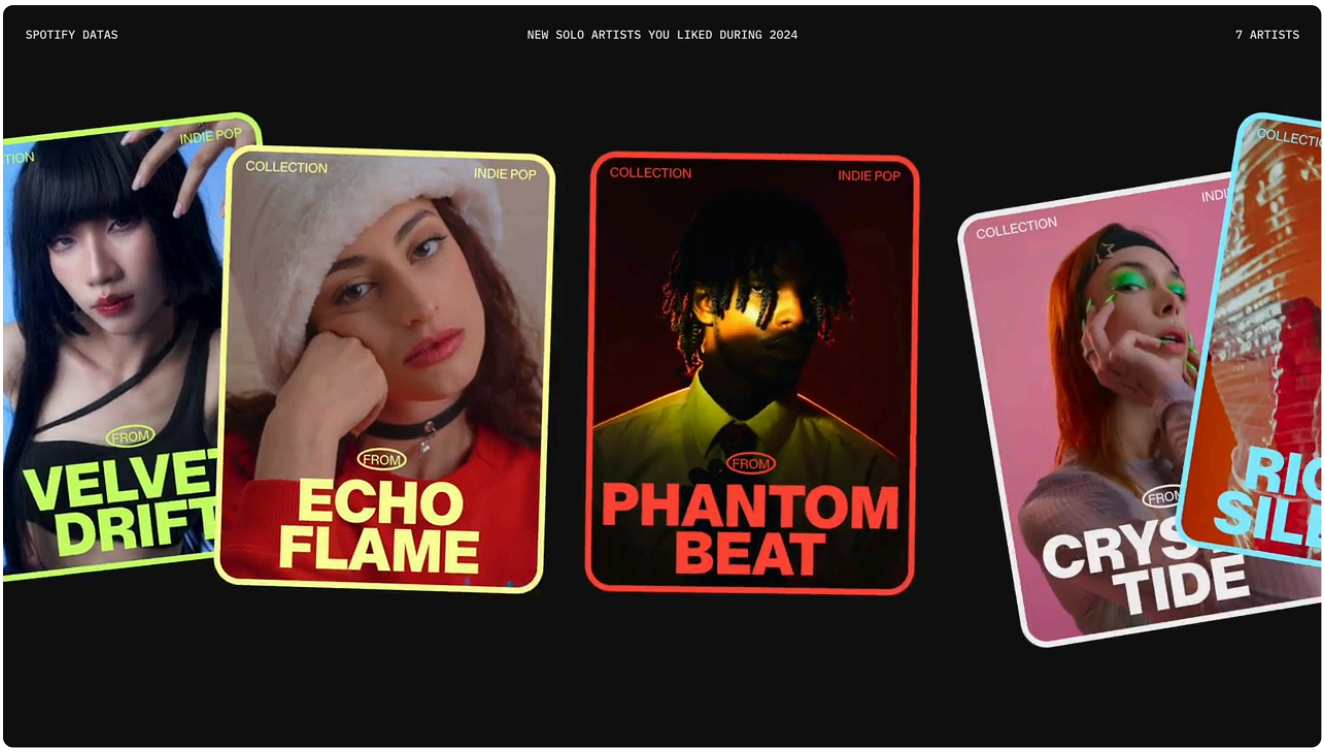
Made With Gsap

001
Card stack



For this effect, I tried to replicate the movement of a stack of cards spreading and sliding across a surface. How does it work? We'll pin a division during a scrolling distance and move its child horizontally at the same time. Let's begin!

HTML Structure

The HTML structure of the effect remains relatively simple. I declared a container as the parent of the section that will hold all my cards.

```
<section class="mwg_effect001">
    <div class="container">
        <div class="cards">
            <div class="card"></div>
            <div class="card"></div>
            <div class="card"></div>
            <div class="card"></div>
        </div>
```

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

Some CSS

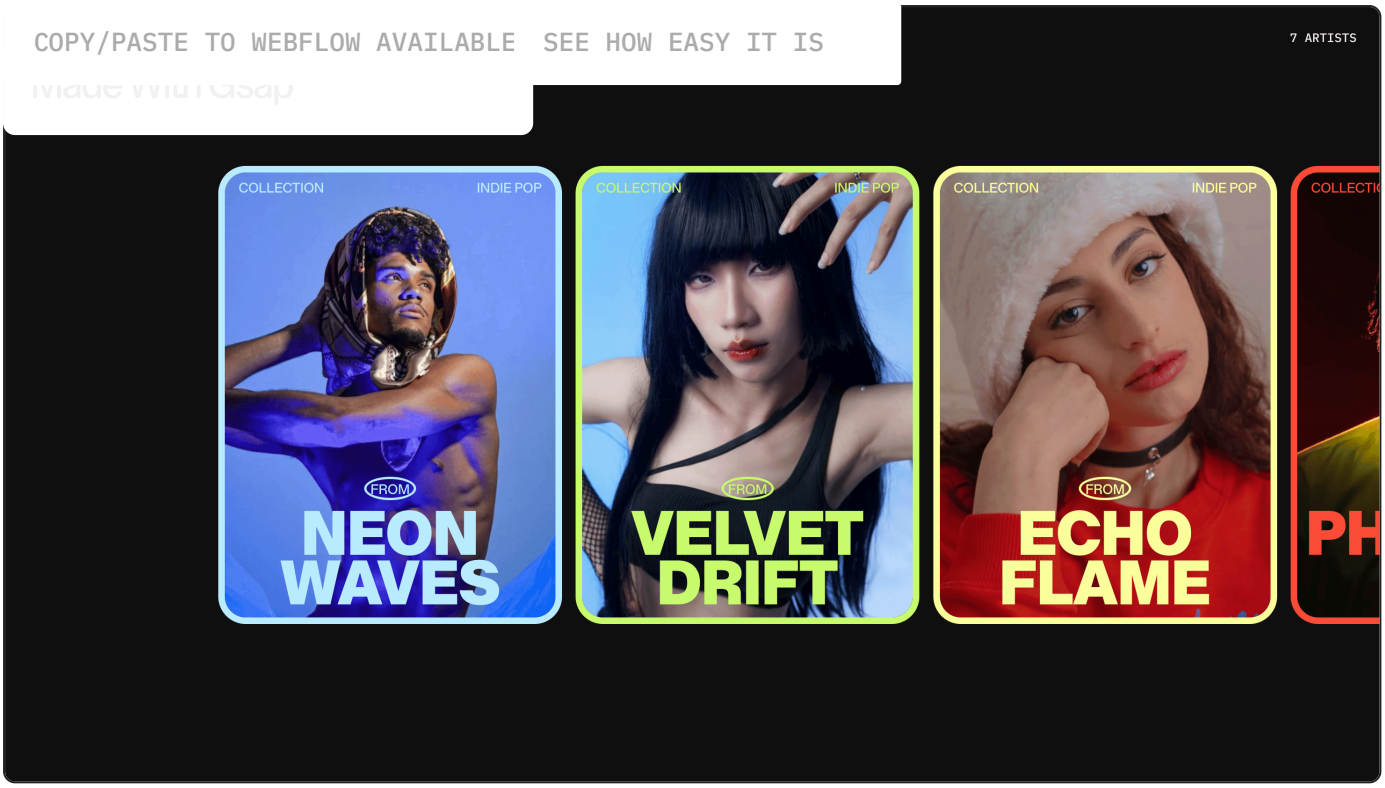
To start, we'll align the cards horizontally and automatically allow the container to expand, even if the cards overflow the viewport. For this, I use the CSS properties `white-space: nowrap` and `width: max-content` combined.

Each card will have its own movement. To optimize its transformations, I use the `will-change: transform` property. This anticipates that its property will change frequently throughout the effect. To prevent a part of the card from being visible before scrolling the container, I add horizontal padding slightly larger than the screen width (to account for the rotation).

```
.mwg_effect001 .cards {
    display: flex;
    gap: 1vw;
    width: max-content;
    white-space: nowrap;
    will-change: transform;
    padding: 0 120vw;
}
```

Here's what you should have at this point:

BATCH 001 — 050 EFFECTS



Moving the container

Let's move this same container from left to right. I start by calculating a movement distance: I subtract the screen width ensuring the effect ends when the container's right edge meets the viewport's right edge.

```
const distance = cardsContainer.clientWidth - window.innerWidth
```

Now, let's use this distance to move our container using the `gsap.to()` method combined with the `scrollTrigger` object, which will link the movement to the scroll. Indeed, the content of the object will define when the animation should be triggered.

```
const scrollTween = gsap.to(cardsContainer, {
    x: - distance,
    ease: 'none', // linear progression
    scrollTrigger: {
        // let's pin our container while our cardsContainer is animating
        trigger: container,
        pin: true,
        scrub: true,
        start: 'top top',
        end: '+=' + distance
```

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

Here, we move the card section along the x-axis while keeping the container fixed so that it stays visible in the viewport using the property `pin: true`. The animation starts when the container reaches the top of the viewport (`start: 'top top'`) and ends after scrolling the set distance, matching natural scroll speed for a smooth feel. For the next steps, note that we store this tween in a variable `scrollTween`.

To go further, we can play around with the `ease` property of this tween. Interesting effects can be achieved using `power1.inOut` or `power2.inOut`, adding a subtle slow-in/slow-out effect.

Moving the cards

To make it dynamic, each card will have a unique effect during its movement. I'll randomize three properties for each card: `xPercent`, `yPercent`, and `rotation`. These properties will shift the card along the x and y axes and rotate it by an angle.

The goal: each card starts with a left, upward shift and rotation. As it moves across the screen, these values are reversed to create visible motion.

Math Mode Activated!
We start by defining a range for each property. For example, to generate a number between 30 and 50:

```
Math.random() * 20 + 30
```

Indeed, `Math.random()` generates a decimal between `0` and `1`. Multiplying by `20` gives a range of `0` to `20`. Adding `30` shifts the range to `30 – 50`.

Then we want to randomly make this value positive or negative. For this, we'll use `Math.random()` again, and retrieve `1` or `-1` with this calculation:

```
Math.random() < 0.5 ? 1 : -1
```

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS
Made With Gsap

`0` and `1`. If the value is less , it returns `-1`.

So, to generate a value between `30 – 50` or `-30 – -50`, we'll assemble those calculation and get this:

```
(Math.random() * 20 + 30) * (Math.random() < 0.5 ? 1 : -1)
```

Why not just use a range of `-50` to `50`? This ensures enough movement. For instance, a small value like `-2` wouldn't be visually noticeable.

Finally, I tween each card using the `gsap.fromTo()` method.

```
const values = {
    // get a value between 30 and 50 or -30 and -50
    x: (Math.random() * 20 + 30) * (Math.random() < 0.5 ? 1 : -1),
    // get a value between 10 and 16 or -16 and -10
    y: (Math.random() * 6 + 10) * (Math.random() < 0.5 ? 1 : -1),
    // get a value between 10 and 20 or -10 and -20
    rotation: (Math.random() * 10 + 10) * (Math.random() < 0.5 ? 1 : -1)
};

gsap.fromTo(card, {
    // let's start from this 3 values
    rotation: values.rotation,
    xPercent: values.x,
    yPercent: values.y
}, {
    // and finish to its 3 opposite values
    rotation: - values.rotation,
    xPercent: - values.x,
    yPercent: - values.y,
    ease: 'none', // linear progression
    scrollTrigger:{
        trigger: card,
        // our tween will listen to our scrollTween container position
        containerAnimation: scrollTween,
        start: 'left 120%' ,
        end: 'right -20%',
        scrub: true // the animation progress with the scroll
    }
})
```

We get three values from the `values` object.We position each card with its own values and move it to the opposite of those values: we start from one value when the card enters

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

value when the card leaves the animation begins when the card's left edge reaches 120% of the viewport and stops when its right edge reaches -20%.

I use `ease: none` to keep the animation linear and aligned with the scroll.

Remember, I stored the container's tween in a variable. We set `containerAnimation: scrollTween` to make the card animate based on the container's position.

Go further

To go further, we can play with the different ranges in the `values` object for more or less pronounced movements. We can also set a numeric value for the `scrub` property to add a delay between when the scroll stops and when the card's movement stops. I recommend using a small `scrub` value, like `0.4`, to avoid confusing the user.

In the example, I used the Lenis smooth scroll to further enhance the fluidity and sliding effect. If you'd like to apply it too, check out our Go Further page to see how to add it easily.

Final code

HTML CSS JAVASCRIPT

```
<section class="mwg_effect001">
    <p class="scroll">Scroll</p>
    <div class="container">
        <div class="header">
            <p>Spotify datas</p>
            <p>New solo artists you liked during 2024</p>
            <p>7 artists</p>
        </div>
        <div class="cards">
            <div class="card">
                <img src="assets/medias/1.jpg" alt="">
                <div class="card-content">
                    <p><span>Collection</span> <span>indie pop</span></p>
                    <div>
                        <span class="from">From</span>
                        <h2>Neon Waves</h2>
```

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

```
<div class="card">
    <img src="assets/medias/2.jpg" alt="">
    <div class="card-content">
        <p><span>Collection</span> <span>indie pop</span></p>
        <div>
            <span class="from">From</span>
            <h2>Velvet Drift</h2>
        </div>
    </div>
</div>
<div class="card">
    <img src="assets/medias/3.jpg" alt="">
    <div class="card-content">
        <p><span>Collection</span> <span>indie pop</span></p>
        <div>
            <span class="from">From</span>
            <h2>Echo Flame</h2>
        </div>
    </div>
</div>
<div class="card">
    <img src="assets/medias/4.jpg" alt="">
    <div class="card-content">
        <p><span>Collection</span> <span>indie pop</span></p>
        <div>
            <span class="from">From</span>
            <h2>Phantom Beat</h2>
        </div>
    </div>
</div>
<div class="card">
    <img src="assets/medias/5.jpg" alt="">
    <div class="card-content">
        <p><span>Collection</span> <span>indie pop</span></p>
        <div>
            <span class="from">From</span>
            <h2>Crystal Tide</h2>
        </div>
    </div>
</div>
<div class="card">
    <img src="assets/medias/6.jpg" alt="">
    <div class="card-content">
        <p><span>Collection</span> <span>indie pop</span></p>
        <div>
            <span class="from">From</span>
            <h2>Riot in Silence</h2>
        </div>
    </div>
</div>
<div class="card">
    <img src="assets/medias/7.jpg" alt="">
    <div class="card-content">
        <p><span>Collection</span> <span>indie pop</span></p>
```

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

```
Made With Gsap                    <br />Flame</h2>
                    </div>
                </div>
            </div>
        </div>
    </div>
</section>
```

Wanna learn something else?

BACK TO THE COLLECTION

Never miss what's next

Your email

SUBMIT

By submitting your email, you'll be the first to know about upcoming updates for Made With Gsap. You can unsubscribe at any time.

SOCIAL

X(TWITTER)
INSTAGRAM
BLUESKY
LINKEDIN

PAGES

HOME
COLLECTION
FAQ
PRICING

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

SUPPORT

Made With Gsap

2025©

PERMISSIONS & TERMS

PRIVACY POLICY